Argus

A Reliable Harness for Long-Horizon Autonomous Research

Argus Team

July 2026 • Technical Report 0.1

This report describes an engineering system for reliable, auditable, long-horizon autonomous research. It documents design intent, architecture, and reliability mechanisms, and it states the current evidence status honestly. It is not a results paper and does not claim state-of-the-art performance, a finished research program, or an accepted autonomous publication.

Contents

1 Abstract

Argus is a harness for running a capable coding-and-reasoning model as an autonomous research agent over long horizons — hours to days — against real, public benchmarks. Its central claim is not that autonomy *succeeds*, but that autonomy can be made *reliable and auditable*: a mission either makes verifiable forward progress under a bounded budget, is explicitly surfaced as blocked, or is judged complete by a reviewer against a structured checklist — never left to spin indefinitely, and never silently "finished" without evidence.

The system rests on one design commitment: **research judgment belongs to the agent; the harness is a domain-agnostic pipe**. Four persistent roles — Manager, Planner, Engineer, and Reviewer — carry every scientific decision (what to try, whether a baseline is strong enough, whether an experiment is finished, whether to submit). The harness underneath them does only domain-agnostic work: budget and rate limiting, persistence and working memory, scheduling and daemon lifecycle, structured input/output, and anti-cheat guardrails. The harness never uses keyword or regex heuristics to second-guess the agent about the substance of the research.

This report documents that commitment concretely. It describes the four-role architecture, in which the Reviewer is the *sole* authority on completion; the reliability mechanisms that keep a long-running session from degrading (a reviewer-curated working-memory checkpoint, bounded session rolls, structured decision-progress classification with a safe round-boundary budget, and supervised background waits); and an evidence methodology that separates reproducible in-repository artifacts from operator-observed preliminary observations, external baselines, and ongoing programs with no public claim.

We are deliberate about what this release does *not* assert. Argus does not claim clean, publishable Argus scores on the benchmark-reproduction programs it is actively running (KernelBench, nanoGPT speedrun, nanochat), and it does not claim to have produced an accepted paper through its autonomous pipeline. External reference numbers appear only as named third-party targets for comparison, never as Argus's own measured results. The contribution here is an engineering substrate for reliable long-horizon autonomy, presented honestly at **Technical Report 0.1** maturity.

2 Introduction

Language-model agents can now read code, run experiments, and write prose with little supervision. A large body of work has shown that structured reasoning-and-acting loops [10], self-reflection over past attempts [7], open-ended skill acquisition [8], and purpose-built agent–computer interfaces for software engineering [9] materially raise what a single agent can do. More recently, systems have attempted the full arc of scientific work — ideation, experimentation, and manuscript writing [4] — and external efforts have reported progress toward automating AI research itself [6]. The frontier is no longer whether an agent *can* take a useful action; it is whether an agent can be trusted to take thousands of them, unattended, without the run quietly falling apart.

That is the problem Argus targets. Give a capable model one hard objective and enough wall-clock time, and the failure mode is rarely a failure of reasoning. It is a failure of the *execution substrate*. We have repeatedly observed four distinct ways a long-horizon run degrades:

High activity, low decision.
The agent stays busy — re-reading the same files, rewriting the same status note, re-polling a healthy job — while making no decision that moves the research forward. Token spend and round count climb; the actual frontier does not. Measuring "progress" by activity is exactly the trap.

Context amnesia.
A single mission's session is resumed dozens or hundreds of times. The underlying model compacts its context lossily on each overflow, discarding working memory, so the agent forgets what it already tried and re-treads dead ends. This is a structural property of unbounded session growth, not a prompt-quality problem.

Background-job babysitting.
The agent launches a long experiment — a training run that already has its own health monitor — and then blocks the main line of work polling it every round, instead of advancing independent tasks while it waits.

Premature acceptance.
The agent declares victory. Without an independent authority checking the claim against evidence, "done" becomes a self-report, and a self-report is not a result.

Layered on top of these is an **integrity** risk that is unique to autonomous benchmarking: an agent optimizing a metric will, if permitted, exploit the harness rather than solve the task — hardcoding an answer, memorizing a fixed evaluation input distribution, or editing a validator to report success. An autonomous research system that cannot resist this is not merely unreliable; it is actively misleading.

None of these five problems is solved by making the model smarter or the prompt longer. They live in the substrate, so they must be fixed in the substrate. But the substrate must fix them *without* pretending to know better than the agent about the research. A harness that starts guessing — via keywords or regexes — whether an objective is "really" a paper task, whether an idea is good enough, or whether a result should count, has simply moved the unreliability somewhere harder to see. Argus's guiding constraint is therefore narrow and strict: the harness may only enforce domain-agnostic invariants (budgets, persistence, scheduling, structured I/O, and anti-cheat), and every scientific judgment must be returned to the agent.

This is an engineering and design report at version 0.1. It documents what Argus is built to do and how, grounded in the current source tree of the `argus-skill` repository [1]. It reports evidence status honestly and conservatively; where a program is ongoing, it says so, and it does not convert an external baseline or a preliminary observation into an Argus result.

The remainder of the report proceeds as follows. Section 3 states the design philosophy that everything else follows from. Section 4 describes the four-role architecture. Section 5 details the reliability mechanisms. Section 6 lays out the evidence tiers and measurement discipline. Section 7 surveys the research programs and the operator workbench. Section 8 is candid about limitations and responsible operation, and Section 9 closes with a roadmap.

3 Design Philosophy

Argus is built around a single sentence, from which the rest of the system is a set of corollaries:

The harness is not smarter than the agent. Scientific judgment — whether an idea is worth trying, whether a baseline is strong enough, why an experiment is slow, whether the work is done — always belongs to the agent. The harness does only domain-agnostic plumbing: budget, persistence, scheduling, structured I/O, and anti-cheat guardrails.

3.1 Two kinds of work, kept apart

Every capability in Argus is classified as either *research judgment* or *domain-agnostic plumbing*, and the two are never allowed to blur.

Research judgment is anything whose correct answer depends on the content of the science: is this direction novel, is this comparison fair, is this measurement trustworthy, is the objective satisfied. All of it is delegated to the agent roles (Section 4). *Domain-agnostic plumbing* is anything whose correct answer does not depend on the science: how many dollars have been spent, where an artifact is stored, when the next mission may start, whether output parses, whether a credential leaked into a log. All of it lives in the harness, and the harness is deliberately "dumb": a thick pipe that feeds tasks in, stores artifacts, and paces work against a budget.

3.2 Corollaries the codebase actually enforces

The commitment is not a slogan; it is visible in the code as a series of deletions and refusals.

No keyword classifiers on the agent's behalf.
Routing an operator's free-text message to chat-versus-task is a single low-effort *model* call, not a character-length heuristic or a bilingual regex. The historical `is_conversational` keyword guesser was removed; the router now biases toward running the full pipeline whenever the model does not explicitly say "chat."

No harness override of a verdict.
A prior version of the reviewer path contained a function that scanned the engineer's summary with regexes and forcibly rewrote a `done` verdict into `continue`. That function and its pattern constants were deleted. The constraint it tried to express — do not accept "done" without execution evidence — now lives in the reviewer's own prompt, judged by the reviewer, not imposed after the fact.

No prose-sniffing for task type or scope.
Whether a mission is a paper task, whether it should run forever, and whether it is a final-submission certification are decided by *explicit structured signals* — a parsed vertical's completion gate, a configuration flag, a backlog tag threaded through to the reviewer — not by grepping the objective text.

Recency, not relevance-guessing, for memory.
The working-memory prelude injected before a mission surfaces the most recent journal entries and marks them non-authoritative advisory. It no longer scores "relevance" by word overlap; which past work matters is the agent's call to make after reading.

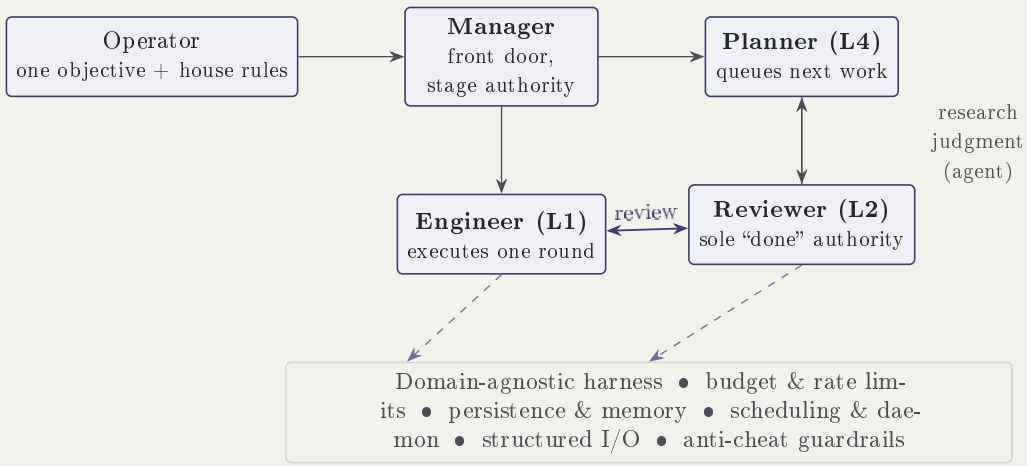
3.3 The one legitimate exception: anti-cheat

There is exactly one place where the harness is *allowed* to constrain the agent's behavior with hard rules, and it is not about research quality — it is about integrity. Because an autonomous agent optimizing a metric will exploit the harness if it can, Argus enforces domain-agnostic anti-cheat invariants: use real public benchmark data, do not reward-hack a fixed evaluation input distribution, keep an exact audit trail of every round, and scrub credentials from artifacts before they are recorded. These guardrails never judge whether the science is good; they only ensure the numbers are honest. That asymmetry — maximal deference on judgment, zero tolerance on integrity — is the philosophical core of the system.

3.4 Reliability over guaranteed success

Finally, the philosophy sets a modest and precise success criterion. Argus does not promise that an autonomous run will produce a state-of-the-art result or an accepted paper; no honest harness

Figure 1. Schematic: four research roles over a domain-agnostic harness (structural diagram, not measured data)



can promise that, because the quality of the outcome is bounded by the agent's judgment and the difficulty of the problem. What Argus promises is that a run will be *reliable and auditable*: bounded in cost, resistant to silent degradation, honest about its evidence, and completed only when an independent reviewer certifies it against a checklist. The rest of this report is the mechanism behind that promise.

4 The Four-Role Architecture

Argus runs four persistent roles. Three of them — Planner, Engineer, and Reviewer — form a numbered execution spine (L4, L1, L2). The fourth, the Manager, deliberately sits outside that numbering because it spans the whole pipeline rather than occupying a station on it. A fifth role, the Curator, is optional and runs only in team/subagent modes. The schematic in Figure 1 shows how the roles sit over the domain-agnostic harness.

4.1 Manager — front door and stage authority

The Manager is the single entry point for the operator. When free-form text arrives, one grounded model call — not a keyword match — decides whether it is chat or a task, selects the vertical (task shape) or authors a new data domain, and commits that choice durably so the rest of the engine trusts it without re-classifying. The Manager is also the *sole* authority for pipeline-stage transitions: it owns the `current_stage` field, and Planner, Engineer, and Reviewer may only *advise* a stage change, never write it themselves. It has its own backend and model configuration and appears alongside the other three roles in the operator's cockpit. Crucially, the Manager never judges whether a result is a win and never runs planning loops — it divides the task and hands the current stage to the Planner.

4.2 Planner (L4) — forward scheduling

The Planner is the continuous scheduler. When the backlog empties in continuous mode, it proposes the next unit of work. It also provides a completion safeguard: if it prematurely reports a project "done" while a full-pipeline completion has not yet been certified, the supervisor *replaces* that verdict with an explicit `scope:final_submission` certification task and hands it back to the Reviewer. This safeguard is driven by an explicit configuration flag, not by inspecting the objective text. The retired L3 "critic" round-polishing layer no longer exists; all acceptance

flows to the Reviewer.

4.3 Engineer (L1) — one round of real work

The Engineer executes a single round against real hardware and real tools: literature search, code, experiments, or manuscript writing. It shares a working directory with the Reviewer, so the Reviewer can inspect exactly what was produced. The Engineer's round loop is where most reliability machinery lives (Section 5): backend-failure recovery, session continuity, context-pressure handling, and the background-subagent cadence wait. The Engineer's prompt is kept intentionally light — the current task, the original objective, the matched or freshly distilled skill, the Reviewer's concrete next step, and turn discipline — so it is not buried under boilerplate.

4.4 Reviewer (L2) — the sole authority on "done"

The Reviewer is the completion authority, and there is no separate hardcoded completion gate anywhere in the system. Each round it inspects the Engineer's output against a structured stage checklist and returns one of three verdicts: `done` (with an optional certification of a verified achievement), `continue` (with a concrete next step for the Engineer), or `blocked` (with the binding constraint). Because the Reviewer runs in the same work-tree and is given the mission's execution log with grep recipes, it can audit *how* a result was produced — catching a hardcoded answer, a skipped step, or a faked metric — rather than trusting the Engineer's summary. The Reviewer also authors the curated working-memory checkpoint and labels each round's decision-progress class, both described in Section 5.

Premature acceptance is a defining failure of autonomous agents. Argus answers it structurally: completion is a *single* structured verdict from a role that sees the artifacts and the execution log, not a self-report from the role that did the work, and not a keyword the harness fished out of a summary. If the Reviewer is wrong, that is a model-quality limitation (Section 8) — but the authority is never ambiguous.

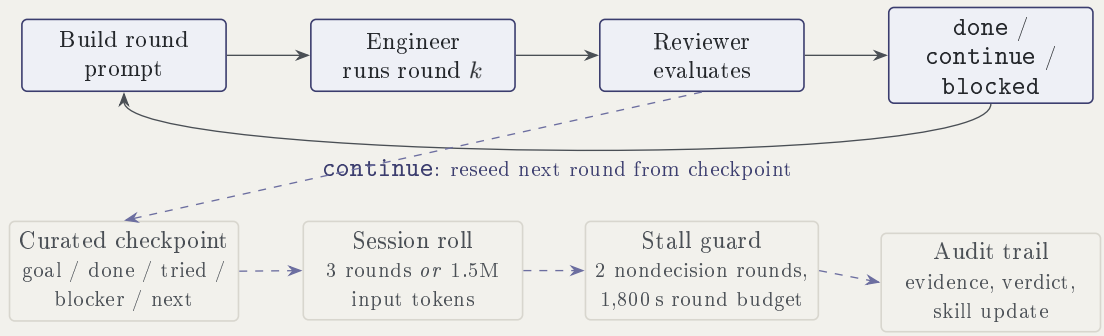
4.5 The harness beneath the roles

Underneath the four roles sits the domain-agnostic core: budget and pricing, persistence and paths, structured I/O ports, daemon locks, and the single anti-stuck mechanism (regime-jump), which detects saturation with a dumb counter, asks the Planner to judge whether to jump, and enforces a never-cleared forbidden ledger — failing soft to a no-op if anything goes wrong. Per-task shapes live in pluggable *verticals* (nanochat, nanoGPT speedrun, KernelBench, research, and others), each supplying its role banner, completion gate, and strategy pool. The outer `LifeSupervisor` is the 7×24 loop: it claims backlog items, injects a non-authoritative memory prelude without polluting the raw objective, runs the mission, accounts cost against budget gates, and writes the journal. State is persisted per project under a global root, so a daemon can be stopped, upgraded, and resumed at a clean mission boundary without re-invoking the Manager.

5 Reliability Mechanisms

The biggest risk in a long-horizon run is execution drift, not weak reasoning. Argus therefore ships a set of guards that keep a mission bounded and honest — each one domain-agnostic, each one refusing to make a scientific judgment itself. Figure 2 sketches how they wrap a single mission's engineer/reviewer loop.

Figure 2. Schematic: one mission's bounded engineer/reviewer loop with context and stall guards (control flow, not measured data)



5.1 Curated working-memory checkpoint

A single mission's session is resumed round after round, and left unchecked it grows into millions of tokens and is compacted lossily many times, erasing working memory and driving the amnesia loop described in Section 2. Argus fixes this without a watchdog. The Reviewer — acting as a memory auditor — maintains a small, *hard-capped* checkpoint: goal, done items, tried-and-failed attempts, a "maturing" list for promising-but-unproven directions, an open blocker, and the next step. The caps are enforced in code, not merely requested in a prompt (for example, at most a handful of done and tried items, each a short line), because the cap *is* the forgetting mechanism: deleting stale detail is the antidote, and the ground truth always remains on disk in the artifacts. Each round the Engineer proposes a handoff; the Reviewer validates it against evidence and writes the next canonical checkpoint. The runner renders that checkpoint as a "curated working memory" block at the top of the next Engineer prompt. If the Reviewer ever mis-writes the checkpoint, the runner keeps the previous one — it never blanks memory.

5.2 Bounded session roll

The checkpoint is only half the fix; the session itself must stay short-lived. A thread is proactively dropped and reseeded from the checkpoint into a *fresh* session after **3 consecutive rounds** (the default `shift_round_limit`) *or* once the previous round's input reaches **1.5M tokens** (the default `thread_token_limit`). Both are configurable through environment variables, and either can be disabled by setting it to zero. The round-count roll bounds context within a single mission; the token-count roll catches an inherited, already-bloated thread on its first round, because a thread resumed across many short missions can reset the round counter before it ever fires. With per-session context bounded, the runaway hundred-compaction spiral cannot occur, and the existing context-pressure and poisoned-session cleanup paths now reseed from the checkpoint — rebirth rather than amnesia.

5.3 Structured decision-progress classification

To distinguish real progress from busy churn, the Reviewer labels each round with a `progress_class`: `decision` or `evidence` (genuine forward motion), or `setup_only`, `artifact_sync_only`, or `none` (nondecision work). The harness counts these structured labels; it never infers progress from activity. **Two consecutive** nondecision rounds trip a stall counter (the default `stall_threshold`). This is paired with a safe **1,800-second** round-boundary budget measured from the last decision or evidence increment. The budget is checked only *between* rounds — it never interrupts a live provider call — so it can end a mission that is spinning without ever severing useful in-flight work. A separate soft/hard round-limit escalation catches the rarer case of a

mission that makes evidence progress every round but never passes its gate, asking the Reviewer to return `blocked` on an external dependency and, failing that, force-ending as `blocked` so the Planner re-plans.

5.4 Dynamic review cadence

Reviewing every round is the default, but it is occasionally wasteful: when the Engineer has just landed a real increment and the next step is unambiguous, a Reviewer round would only restate it. In that case the Engineer may append a single `CONTINUE_WORK: <next step>` line to request one deferred review. It may defer only once in a row; the last available round always returns to the Reviewer; and `done` can still come only from the Reviewer. The harness does not guess from prose whether to skip — it responds only to the explicit request.

5.5 Background-subagent cadence wait

When the Engineer starts a self-monitored long-running job — a GPU training run that already has its own supervisor checking health, early-stopping on failure, and reporting a terminal result — babysitting it every round wastes budget. Argus reads the subagent registry and classifies in-flight jobs as self-watched or needs-attention. If the Engineer explicitly replies `WAIT_FOR_SUBAGENT: <task_id>` and that id matches a healthy self-watched job, the runner skips the expensive Reviewer round and sleeps on the job's own monitoring cadence (clamped to a safe 30–900 s), waking early when the job finishes. The wait is the agent's explicit choice, not the harness deciding for it; a stale or mismatched sentinel is simply ignored.

5.6 Live credential guard

Finally, a domain-agnostic secret guard scrubs newly written artifacts for credentials before they reach the event log or the Reviewer's prompt, and redacts persisted event and usage copies. Redaction never judges scientific quality; when it rewrites an artifact it tells the Reviewer so provenance hashes can be rebuilt, and a scan error or oversized artifact explicitly blocks unconditional certification rather than silently passing.

None of these mechanisms decides whether the research is good. Each one enforces a domain-agnostic invariant — bounded context, counted decisions, a safe budget, a supervised wait, a scrubbed artifact — and every one fails soft, preserving the agent's memory and judgment when anything goes wrong. That is the difference between a harness that makes autonomy reliable and one that quietly takes the science out of the agent's hands.

6 Evaluation Methodology

This report does not present a results table of Argus scores, because doing so honestly would require reproducible, audited evidence that the current release does not yet have for its ongoing programs. Instead, this section documents the *methodology* — how Argus separates kinds of evidence, and what measurement discipline it applies — so that any number a reader later encounters can be placed in the correct tier.

6.1 Evidence tiers

Argus classifies every quantitative statement into one of four tiers. The tier, not the magnitude, determines what the number is allowed to claim.

Tier	What it means	What it may claim
Reproducible release evidence	An in-repository artifact — raw scored rows, logs, manifests, and a build script — that a third party can regenerate from a named commit.	An Argus result for that specific run, on stated hardware.
Operator-observed preliminary	A number seen during a live run but not yet packaged as a reproducible artifact (e.g. a mid-run log).	A preliminary observation only; not a result, not a claim.
External baseline	A published third-party target from other work, used for comparison.	A reference target; explicitly *not* an Argus achievement.
Ongoing / no public claim	A program actively running without certified, reproducible completion.	Status "ongoing"; no numeric Argus claim.

Table 1: The four evidence tiers. A number's tier is fixed by its provenance, and labels travel with the number wherever it appears.

6.2 Measurement discipline

Three rules govern how measurements are produced and reported.

Same-hardware, same-harness baselines.
A comparison is only fair when the baseline is reproduced on the same hardware and through the same measurement harness as the Argus run, with hardware and metric caveats stated explicitly. Cross-hardware numbers are labeled, not silently compared.

Anti-cheat by construction.
Evaluation uses real public benchmark data. Where a metric could be gamed by memorizing a fixed evaluation input distribution, the input is randomized per evaluation so a solution cannot hard code statistics of a known distribution — a fast-because-faked kernel is designed to be impossible to pass off as genuinely fast. Every round's evidence, verdict, and skill update is persisted for audit.

Reviewer-certified completion.
A program is "complete" only when the Reviewer certifies it against the full-pipeline stage checklist. There is no hardcoded completion gate and no independent validator that can mark a program done; the structured verdict is the single source of truth.

6.3 Evidence map

The claims in this report are grounded in the current `argus-skill` repository [1] rather than in ephemeral run directories. Readers who want to verify the mechanisms can consult stable in-repository sources: the architecture map in `docs/ARCHITECTURE.md`; the maintainer guide in `AGENTS.md`; the Engineer round loop, session-roll, and stall guards in `argus_skill/engineer/runner.py` and `argus_skill/engineer/checkpoint.py`; the completion authority and progress classes in `argus_skill/reviewer/_core.py` and its `reviewer_schema.json`; the Manager front door and stage authority in `argus_skill/manager/`; the 7×24 scheduler in `argus_skill/life/supervisor/`; and the control-plane reliability record in `docs/experiment/2026-07-12-completion-livelock/`, whose experiments verify that four control-plane omissions no longer cause a stuck stage or a repeated planning loop, with the

repaired suite reported as passing in that document's ledger. The release lineage for these paths is the repository's committed history on `main`; this report deliberately cites repository paths and commit lineage rather than any transient local filesystem location.

7 Research Programs and Workbench

Argus's day-to-day product is a benchmark-reproduction agent that drives real public benchmarks toward a target under Reviewer judgment. This section surveys the programs it runs and the workbench an operator uses to watch and steer them. Every program below is described with its *status*, and that status is conservative on purpose.

7.1 Programs and their status

KernelBench on B200. A program to write efficient GPU kernels and measure them against a public kernel benchmark [5], including the anti-cheat construction that randomizes evaluation inputs so a kernel cannot hardcode statistics of a known input distribution. Status: **ongoing**; no certified reproducible Argus score is claimed in this release.

nanoGPT speedrun on 8×H100. A program to reproduce a fast GPT-training loop [2] on the same hardware and harness as the baseline, so any wall-clock comparison is like-for-like. Status: **ongoing**.

nanochat on B200. A program to reproduce a small end-to-end ChatGPT-style training pipeline [3] and its validation metric. Status: **ongoing**.

Research-paper pipeline. An optional `research` vertical that drives an idea-to-submission academic pipeline through per-stage checklists and a Reviewer peer-review gate. It is not the default identity of the project, and it is lazy-loaded only when a project's vertical is `research`. Status: **ongoing**; Argus does *not* claim to have produced an accepted paper through its autonomous pipeline.

Math research. An exploratory program applying the same harness to open mathematical questions. Status: **ongoing**; exploratory.

7.2 External reference targets

The following third-party numbers are sometimes used as comparison targets in program documentation. They are **external reference targets from other work**, listed here only to name the bar; they are *not* Argus results and this release makes no claim of having matched them.

This release does not claim clean, publishable Argus scores for the KernelBench, nanoGPT speedrun, or nanochat programs, and it does not claim an accepted autonomous paper. Where a reproducible in-repository artifact exists for a specific run, that artifact — not this report — is the authority for that run's number.

Program (target metric)	External target	Argus claim in this release
KernelBench-style speed-of-light fraction	`0.547 SOL`	None; program ongoing.
nanoGPT speedrun wall-clock	`77.3 s`	None; program ongoing.
nanochat validation bits-per-byte	`0.9109 val_bpb`	None; program ongoing.

Table 2: Named external/reference targets. These are third-party bars used for comparison, tier "external baseline" in Table 1; none is an Argus measured result.

7.3 The operator workbench

Argus is not only a daemon; it is a live workbench. An operator configures the machine's policy once — author identity, per-role backend and model access, GPU allocation — and then works through a cockpit.

Cockpit.
An Ink terminal UI (and a web surface) is the front door. Free text is routed by the Manager; an operator can say "switch to the copilot backend" in plain language and it takes effect without editing a config file.

Mission view.
The four roles — Manager, Planner, Engineer, Reviewer — are shown as peers, and the operator can follow a mission round by round via `-status`, `-watch`, and `-follow`, backed by a structured event stream (`round.main.completed`, `round.review.completed`, `session.roll`, and others).

Inspectable artifacts.
Because the Engineer and Reviewer share a work-tree and every round's evidence and verdict are persisted, the operator can open exactly what was produced and read the Reviewer's reasoning, rather than trusting a summary.

Operator intervention.
The operator can abort a single mission or drain and stop the daemon at a clean boundary; a stop-and-resume preserves the campaign identity so an upgrade does not silently re-plan from a stale state.

Persistence.
Per-project state — backlog, event log, journal, and continuous-mode configuration — lives on disk under a global root, so a run survives restarts and can be audited after the fact.

8 Limitations and Responsible Operation

Argus is a harness that gives an agent real judgment, real tool access, and real persistence. It is not a system that guarantees good science. This section states the limitations plainly, because a report that documents reliability mechanisms without documenting their boundaries would itself be misleading.

8.1 Limitations

Provider dependence.
Argus drives an external agent CLI (Codex, Claude Code, or GitHub Copilot CLI) and

inherits that provider's capability, availability, latency, and pricing. A provider outage, a model regression, or a rate limit is felt directly, and the quality ceiling of any run is the quality ceiling of the underlying model.

Objective quality is the operator's.
The system optimizes the objective it is given. A vague, ill-posed, or trivially gameable objective yields vague or gamed work; Argus enforces integrity of *measurement*, not wisdom of *goal*.

No guarantee of novelty, correctness, or SOTA.
Nothing in the harness guarantees that an idea is novel, that a proof or result is correct, or that a number is state-of-the-art. Those are properties of the agent's judgment and the problem's difficulty, and the harness deliberately does not fake them.

Reviewer fallibility.
The Reviewer is the sole completion authority, and it is a model. A wrong Reviewer can accept an incomplete result or reject a finished one. Argus reduces this risk by giving the Reviewer the shared work-tree, the execution log, and a structured checklist — but it cannot eliminate model error, and there is no independent oracle behind the Reviewer.

Benchmark-specific integrity.
The anti-cheat guardrails are domain-agnostic, but a determined agent optimizing a metric explores the exploit surface of each specific benchmark. New benchmarks can present new exploits (a leaked test set, an unrandomized input, a side channel) that the current guardrails were not written for; integrity is an ongoing engineering obligation, not a solved problem.

Cost.
Long-horizon autonomy consumes provider tokens continuously. Budgets and rate limits bound spend, but 7×24 operation is not free, and a poorly bounded campaign can be expensive.

Secret handling.
The credential guard scrubs artifacts before they are recorded, but it is a domain-agnostic scanner, not a proof. Operators remain responsible for how secrets enter the working environment, and a scan gap or an oversized artifact deliberately blocks unconditional certification rather than guaranteeing zero leakage.

8.2 Responsible operation

Given these limitations, Argus is meant to be operated, not fired and forgotten. Responsible use means: give it well-posed objectives and honest benchmarks; set budgets deliberately; read the Reviewer's reasoning rather than trusting a headline; treat every number according to its evidence tier (Table 1); and publish only what a reproducible in-repository artifact supports. The system is built to make this easy — persisted artifacts, an auditable round-by-round trail, and a single, inspectable completion authority — but the discipline of honest reporting ultimately rests with the operator.

Argus is under active development at Technical Report 0.1. The architecture and reliability mechanisms described here are implemented and grounded in the current source tree; the benchmark-reproduction programs are ongoing and make no certified public Argus claims in this release. Treat any performance number in this repository's documentation as either an explicitly labeled external/reference baseline or a reproducible in-repository artifact for a specific run — never as a general guarantee of capability.

9 Conclusion and Roadmap

Argus is an attempt to make long-horizon autonomous research *reliable* rather than merely possible. Its wager is that the hard part of unattended autonomy is not reasoning but the execution substrate: sessions that lose memory, rounds that burn budget without deciding anything, background jobs that block the main line of work, and self-reported "completion" that no independent authority ever checked. Argus addresses each of these structurally, in a harness that is deliberately dumber than the agent it serves — a domain-agnostic pipe for budget, persistence, scheduling, structured I/O, and anti-cheat — while every scientific judgment stays with the four roles.

The result is a system with a narrow, honest promise. Argus does not guarantee a state-of-the-art number or an accepted paper; it guarantees that a run is bounded in cost, resistant to silent degradation, honest about which tier its evidence belongs to, and completed only when a Reviewer certifies it against a checklist. That is a smaller claim than "autonomous science works," and it is the claim we are willing to stand behind at this maturity.

9.1 Roadmap

Several directions follow directly from the limitations in Section 8:

From ongoing to reproducible.
Convert the KernelBench, nanoGPT speedrun, and nanochat programs from ongoing status into reproducible in-repository evidence artifacts — raw rows, logs, manifests, and build scripts tied to a named commit — so that a specific run can make a specific, auditable Argus claim on stated hardware.

Stronger integrity coverage.
Extend the anti-cheat guardrails as each new benchmark reveals its own exploit surface, and grow the structured stage checklists the Reviewer verifies against, keeping machine checks in the checklist the Reviewer reads rather than in a separate validator that could become the single point of dishonesty.

Reviewer robustness.
Reduce Reviewer error through better execution-log auditing and evidence-grounded verdicts, without ever introducing a hardcoded completion gate behind it.

Cost and observability.
Tighten budget accounting and the operator cockpit so that a 7×24 campaign is cheaper to run and easier to steer at a glance.

Richer reporting figures.
Replace the LaTeX-native schematic panels in this report with rendered architecture and lifecycle figures in a later revision, once the corresponding assets are available.

Argus today is an engineering substrate for reliable, auditable autonomy, offered honestly at version 0.1. The roadmap above is the path from a trustworthy harness to trustworthy *results* — and the discipline that got us here, refusing to let the harness pretend it is smarter than the agent or that a number is better than its evidence, is the discipline we intend to keep.

References

[1] Argus Team. argus-skill: A 7×24 harness for long-horizon autonomous research. GitHub repository, 2026.

[2] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. GitHub repository, 2022.

[3] Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. GitHub repository, 2025.

[4] Chris Lu et al. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[5] Anne Ouyang et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

[6] Recursive. First steps toward automated AI research. GitHub repository, 2026.

[7] Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

[8] Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[9] John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[10] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.